

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..12

CUSTOM CLOUD STORAGE

 Copy as Markdown

Custom cloud storage

Store files on a NAS, a local disk, or another object store when AWS, Google Cloud Storage, or Cloudflare R2 are not the right fit. A custom backend implements `CloudStorage` and is registered with `pod.addCloudStorage` before the server starts. Serverpod does not ship a local filesystem package yet, so this page is the workaround: subclass the default database storage and write bytes to disk.

The [file uploads](#) page covers the default database backend and the GCP, S3, and R2 packages. Use this page when those are not enough.

When to use a custom backend

You want	Use
Files on local disk or a NAS, with <code>FileUploader</code> talking to your API server	This page
MinIO, Synology S3, LocalStack, or another S3 API	A custom S3-compatible endpoint
AWS S3, Google Cloud Storage, or Cloudflare R2	Configure a storage provider

Pick one upload path and stick to it. Mixing a disk backend with a signed provider URL (or the other way around) is why `storeFile` never runs and `FileUploader` returns `false`.

How uploads reach storage

The `FileUploader` never calls `CloudStorage`. It POSTs or PUTs to the URL inside the upload description.



Ask AI

Through your API server (database storage, local disk, NAS):

1. An authorized endpoint calls `session.storage.createUploadDescription`.
2. Storage inserts a short-lived auth row and returns a URL on your API server:
`/serverpod_cloud_storage?method=upload&storage=...&path=...&key=...`.
3. The app POSTs the raw bytes to that URL.
4. The server checks the auth row and calls `storeUnverifiedFile` (not `storeFile`).
5. An authorized endpoint calls `session.storage.verifyUpload`.

Straight to an object store (S3, GCP, R2, MinIO):

1. The storage returns a signed URL at the provider.
2. The app uploads to that URL. The file never passes through Serverpod.
3. Call `verifyUpload` to check that the object exists.
4. The `storeFile` method is not called. Use it only for server-side writes
(`session.storage.storeFile`).

A class that extends `CloudStorage` and returns JSON whose `url` is a local file path never receives the upload. The client has no valid URL to POST to, and even a correct server URL would call `storeUnverifiedFile`, not `storeFile`.

The built-in upload URL only accepts a `DatabaseCloudStorage`. Extend that class, not `CloudStorage`, if the app uses `FileUploader` against your API server.

Store files on local disk

Keep using `FileUploader` against the API server. Extend `DatabaseCloudStorage` so the upload URL stays registered. Keep the inherited `createUploadDescription`, `publicDownloadUrl`, and `temporaryDownloadUrl`. Those create the auth row and the `/serverpod_cloud_storage` URLs. Override every method that reads or writes bytes, including `storeUnverifiedFile` and `retrieveFileWithStat`, which the upload URL calls and which are not on `CloudStorage`.

The database is still required. Upload auth and temporary download tokens live in tables; the file bytes go to disk.

```
lib/src/cloud_storage/local_storage.dart
```

```
import 'dart:io' as io;
import 'dart:typed_data';

import 'package:path/path.dart' as p;
import 'package:serverpod/serverpod.dart';
```

```

class LocalStorage extends DatabaseCloudStorage {
  LocalStorage({
    required String storageId,
    required String pathPrefix,
  }) : pathPrefix = p.normalize(p.absolute(pathPrefix)),
       super(storageId);

  final String pathPrefix;

  io.File _file(String path) {
    final resolved = p.normalize(p.join(pathPrefix, path));
    if (!p.isWithin(pathPrefix, resolved)) {
      throw CloudStorageException(
        'Path "$path" is outside storage "$storageId".',
      );
    }
    return io.File(resolved);
  }

  Future<void> _write(
    String path,
    ByteData byteData,
    StoreFileOptions options,
  ) async {
    if (options.expiration != null) {
      throw CloudStorageUnsupportedOperationException(
        storageId: storageId,
        operation: 'per-file expiration',
      );
    }

    final file = _file(path);
    if (options.preventOverwrite && file.existsSync()) {
      throw CloudStorageFileAlreadyExistsException(
        storageId: storageId,
        path: path,
      );
    }

    file.parent.createSync(recursive: true);
    await file.writeAsBytes(
      byteData.buffer.asUint8List(
        byteData.offsetInBytes,
        byteData.lengthInBytes,
      ),
    );
  }

  @override
  Future<void> storeFile({
    required Session session,
    required String path,
    required ByteData byteData,
    StoreFileOptions options = const StoreFileOptions(),
  }) async {
    await _write(path, byteData, options);
  }
}

```

```

@Override
Future<void> storeUnverifiedFile({
    required Session session,
    required String path,
    required ByteData byteData,
    StoreFileOptions options = const StoreFileOptions(),
}) {
    return storeFile(
        session: session,
        path: path,
        byteData: byteData,
        options: options,
    );
}

@Override
Future<ByteData> retrieveFile({
    required Session session,
    required String path,
}) async {
    final file = _file(path);
    if (!file.existsSync()) {
        throw CloudStorageFileNotFoundException(
            storageId: storageId,
            path: path,
        );
    }
    final bytes = await file.readAsBytes();
    return ByteData.sublistView(bytes);
}

@Override
Future<({ByteData file, FileStat stat})> retrieveFileWithStat({
    required Session session,
    required String path,
}) async {
    final bytes = await retrieveFile(session: session, path: path);
    return (
        file: bytes,
        stat: await statFile(session: session, path: path),
    );
}

@Override
Future<FileStat> statFile({
    required Session session,
    required String path,
}) async {
    final file = _file(path);
    if (!file.existsSync()) {
        throw CloudStorageFileNotFoundException(
            storageId: storageId,
            path: path,
        );
    }
    final ioStat = file.statSync();
    return FileStat(
        size: ioStat.size,

```

```

        lastModified: ioStat.modified.toUtc(),
    );
}

@override
Future<bool> fileExists({
    required Session session,
    required String path,
}) async {
    return _file(path).existsSync();
}

@override
Future<void> deleteFile({
    required Session session,
    required String path,
}) async {
    final file = _file(path);
    if (file.existsSync()) {
        await file.delete();
    }
}

@override
Future<bool> verifyUpload({
    required Session session,
    required String path,
}) async {
    return _file(path).existsSync();
}
}

```

This example does not persist `FileMetadata` (content type, cache headers, custom keys). Public downloads still get a content type from the file extension. Per-file `expiration` throws `CloudStorageUnsupportedOperationException`. A written file is available immediately, so `retrieveFile` can succeed before `verifyUpload`. The default database storage hides bytes until verify.

Use an absolute directory. A `~` in `pathPrefix` is not expanded.

Register the storage

Call `pod.addCloudStorage` before `pod.start()`. The same `storageId` replaces the default. Register both `public` and `private` if you want neither to stay in the database.

```

import 'dart:io';
import 'package:path/path.dart' as p;

```

lib/server.dart

```

pod.addCloudStorage(
  LocalStorage(
    storageId: 'public',
    pathPrefix: p.join(Directory.current.path, '..', 'storage', 'public'),
  ),
);
pod.addCloudStorage(
  LocalStorage(
    storageId: 'private',
    pathPrefix: p.join(Directory.current.path, '..', 'storage', 'private'),
  ),
);

```

The public download URL is served only for `storageId` `public`. Private files need `temporaryDownloadUrl` or `retrieveFile`. See [Access stored files](#).

If you replace both defaults with a class that extends `CloudStorage` rather than `DatabaseCloudStorage`, Serverpod does not register `/serverpod_cloud_storage`, and `FileUploader` has nowhere to post.

Raise the request size limit

Uploads that post to your API server are capped by two limits. The endpoint uses the smaller of the two.

Limit	Default	Set in
<code>maxRequestSize</code>	524288 (512 KiB)	<code>config/<run-mode>.yaml</code> or <code>SERVERPOD_MAX_REQUEST_SIZE</code>
<code>UploadOptions.maxFileSize</code>	10 MiB	<code>createUploadDescription</code>

A 5 MB file with default config is rejected even though `maxFileSize` is 10 MiB. Raise `maxRequestSize` in every run-mode YAML you use (development, staging, production, and test) to at least the largest file you accept:

```
config/development.yaml
```

```
maxRequestSize: 52428800
```

Object-store uploads (S3, GCP, R2) go to the provider, so `maxRequestSize` does not apply to the file body. See [Size limits](#) on the file uploads page.

Authorize the upload endpoints

The upload URL is a capability token: anyone who has it can POST the file until it expires. Require a signed-in user on the endpoint (`requireLogin`), create the path on the server, and do not accept a client-supplied path that can escape into another user's prefix. See [Authentication basics](#).

```
class FileEndpoint extends Endpoint {
  @override
  bool get requireLogin => true;

  Future<String> getUploadDescription(Session session) async {
    final userIdentifier = session.authenticated!.userIdentifier;

    return await session.storage.createUploadDescription(
      storageId: 'public',
      path: 'uploads/$userIdentifier/avatar.png',
    );
  }
}
```

Use the same authorization on `verifyUpload`. After a successful verify, persist the path your server generated, not a path the client sent.

The rest of the app flow matches [file uploads](#): describe, `FileUploader.upload`, then `verifyUpload`. On failure, `FileUploader` returns `false` with no status code and no exception, so a dropped connection and a file that is too large look the same.

Implement the CloudStorage methods

Throw `CloudStorageException` subclasses on failure, not generic errors.

Method	Role
<code>storeFile</code>	Server-side write. Honor <code>StoreFileOptions.preventOverwrite</code> and <code>metadata</code> , or throw <code>CloudStorageUnsupportedOperationException</code> .
<code>storeUnverifiedFile</code>	Client upload through <code>/serverpod_cloud_storage</code> . Not on <code>CloudStorage</code> ; override it on <code>DatabaseCloudStorage</code> .
<code>retrieveFile</code>	Return bytes, or throw <code>CloudStorageFileNotFoundException</code> .
<code>retrieveFileWithStat</code>	Bytes plus metadata for public and temporary HTTP GET. Not on <code>CloudStorage</code> .

Method	Role
<code>statFile</code>	Size and metadata.
<code>fileExists</code>	True if the file is available. Override it on a <code>DatabaseCloudStorage</code> subclass; the parent queries the database instead of calling <code>statFile</code> .
<code>deleteFile</code>	No-op if the file is missing.
<code>publicDownloadUrl</code>	Public HTTP URL, or <code>CloudStorageUnsupportedOperationException</code> .
<code>temporaryDownloadUrl</code>	Time-limited URL, or unsupported.
<code>createUploadDescription</code>	Return <code>BinaryUploadDescription</code> or <code>MultipartUploadDescription</code> .
<code>verifyUpload</code>	<code>true</code> if the client upload is present.

The `FileUploader` understands two JSON shapes. Extra keys such as `path`, `expiration`, or `maxFileSize` are ignored. Auth and size belong in the database auth row (API-server uploads) or in the signed URL (object-store uploads).

- Binary: `{ "url", "type": "binary", "headers", optional "method", "file-name" }`
- Multipart: `{ "url", "type": "multipart", "field", "file-name", "request-fields" }`

Use an S3-compatible store

If the NAS or appliance speaks S3, do not send the file through `/serverpod_cloud_storage`. Use `serverpod_cloud_storage_s3_compat` with `CustomEndpointConfig`, or extend `CloudStorage` and return a signed `BinaryUploadDescription` with `method: 'PUT'`, the same pattern as native Google Cloud Storage. Then `verifyUpload` checks that the object exists at the provider. `storeFile` is unused for client uploads.

Troubleshooting

Why does `FileUploader.upload` return `false` after a few milliseconds?

The file is larger than the smaller of `maxFileSize` and `maxRequestSize`. With default config that cap is 512 KiB. The connection can reset (`Connection reset by peer`) instead of returning a 413. Raise `maxRequestSize`, then confirm `uploaded` is `true` before calling `verifyUpload`.

Why is `storeFile` never called?

Client uploads do not call it. Override `storeUnverifiedFile`. If the class extends `CloudStorage` instead of `DatabaseCloudStorage`, the built-in upload handler returns `false` and never reaches your code.

Why are public URLs 404s after a successful upload?

The `retrieveFileWithStat` method still reads the database unless you override it. Inherited `publicDownloadUrl` is fine once that method and `fileExists` read from disk.

Related

- [File uploads](#): describe, upload, verify, and the GCP, S3, and R2 packages.
- [Authentication basics](#): `requireLogin` on the upload endpoint.
- [Configuration](#): run-mode YAML files where `maxRequestSize` is set.
- [Sessions](#): the `storage` member used in the examples above.